上海科技大学
ShanghaiTech University

CS240 Algorithm Design and Analysis

Lecture 14

NP and Computational Intractability

Fall 2025
2025.11.11

Sequencing Problems

- **Basic genres**
  - Packing problems: SET-PACKING, INDEPENDENT SET
  - Covering problems: SET-COVER, VERTEX-COVER
  - Constraint satisfaction problems: SAT, 3-SAT
  - Sequencing problems: HAMILTONIAN-CYCLE, TSP
  - **Partitioning problems: 3D-MATCHING, 3-COLOR**
  - Numerical problems: SUBSET-SUM, KNAPSACK

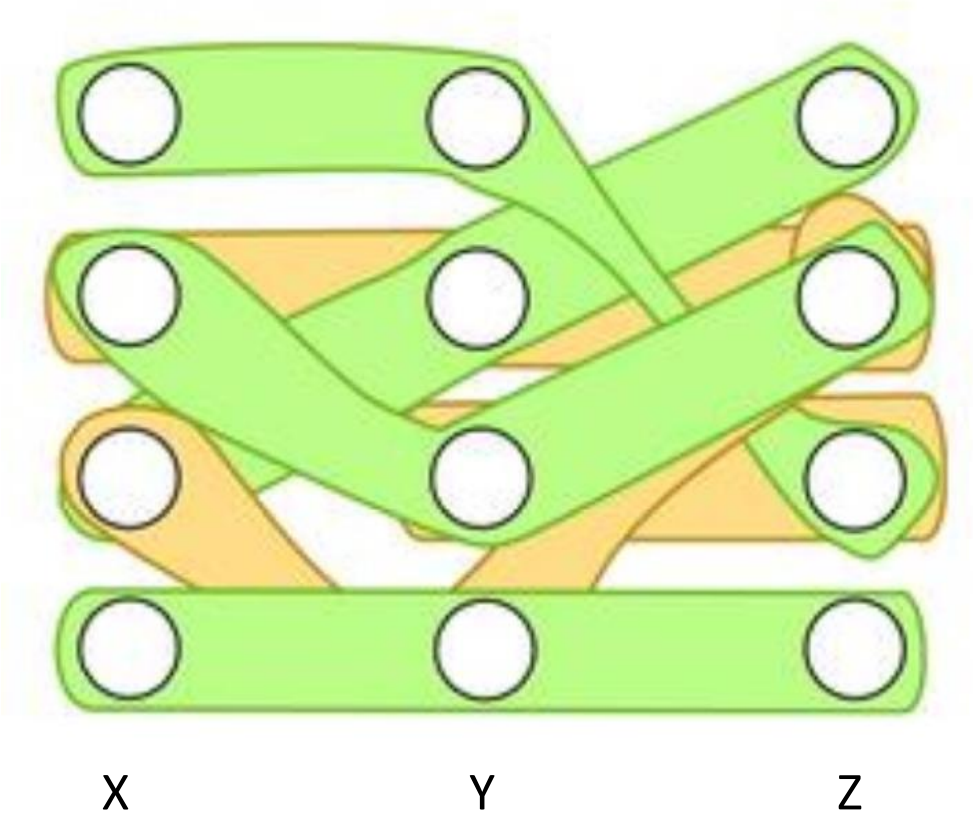
3-Dimensional Matching

- **3D-MATCHING.** Given n instructors, n courses, and n times, and a list of the possible courses and times each instructor is willing to teach, is it possible to make an assignment so that all courses are taught at different times?

Instructor	Course	Time
Wayne	COS 423	MW 11-12:20
Wayne	COS 423	TTh 11-12:20
Wayne	COS 226	TTh 11-12:20
Tardos	COS 523	TTh 3-4:20
Tardos	COS 423	TTh 11-12:20
Tardos	COS 423	TTh 3-4:20
Kleinberg	COS 226	TTh 3-4:20
Kleinberg	COS 226	MW 11-12:20
Kleinberg	COS 423	MW 11-12:20

3-Dimensional Matching

- **3D-MATCHING.** Given disjoint sets X, Y and Z, each of size n and a set $T \subseteq X \times Y \times Z$ of triples, does there exist a set of n triples in T such that each element of $X \cup Y \cup Z$ is in exactly one of these triples?



3-Dimensional Matching

- **3D-MATCHING.** Given disjoint sets X, Y and Z, each of size n and a set $T \subseteq X \times Y \times Z$ of triples, does there exist a set of n triples in T such that each element of $X \cup Y \cup Z$ is in exactly one of these triples?

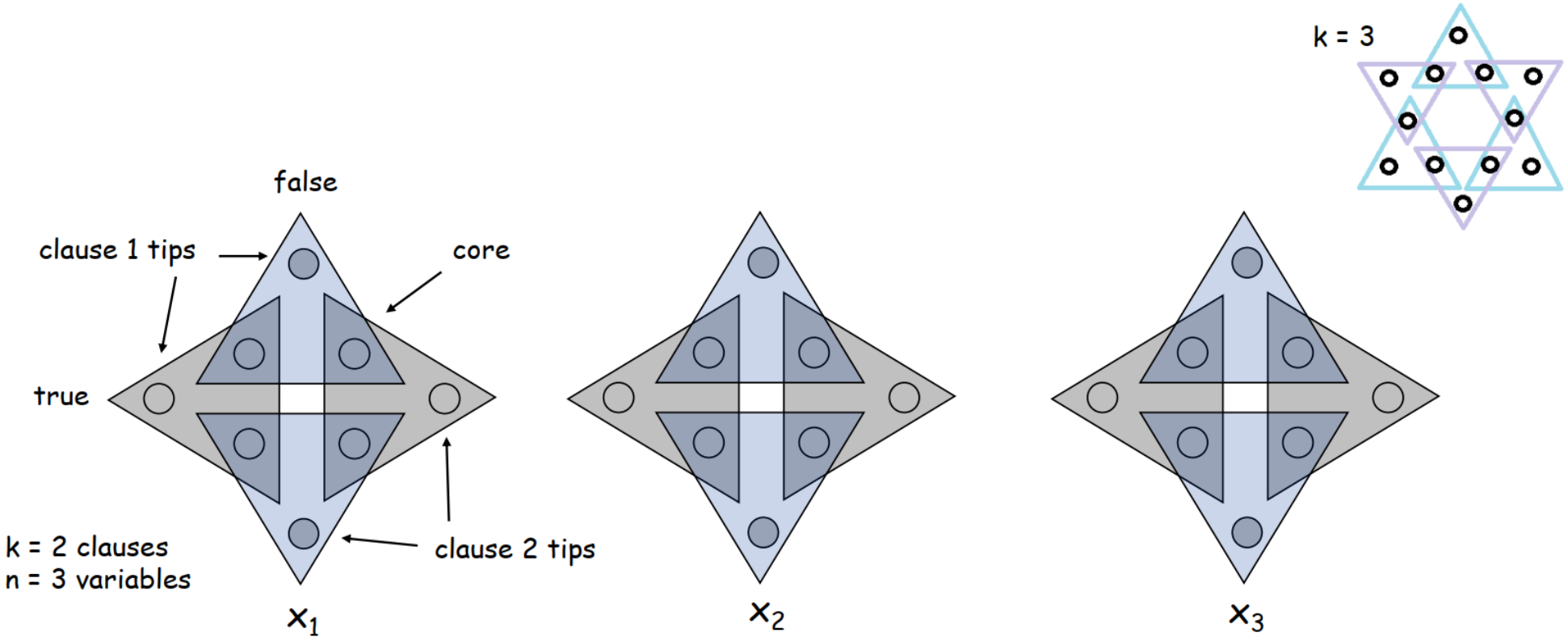
- **Claim.** 3-SAT $\leq_p$ 3D-MATCHING
- **Pf.** Given an instance ϕ of 3-SAT, we construct an instance of 3D-matching that has a perfect matching iff ϕ is satisfiable

- **Recipe to establish NP-completeness of problem Y**
  - Step 1. Show that Y is in NP
  - Step 2. Choose an NP-complete problem X
  - Step 3. Prove that X $\leq_p$ Y

3-Dimensional Matching

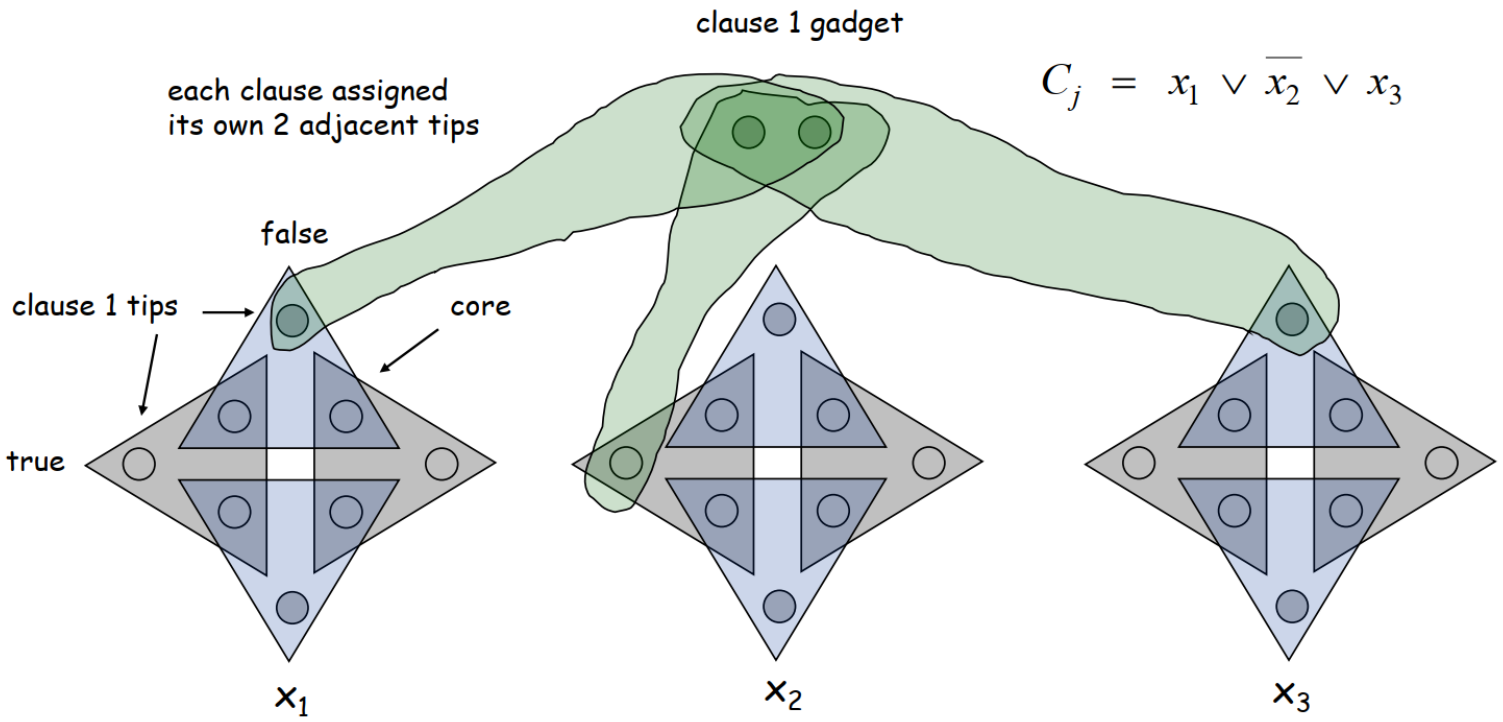
Number of clauses

- **Construction. (part 1)**
  - Create gadget for each variable x_i with 2k core elements and 2k tip elements.
  - No other triples will use core elements
  - In gadget i, 3D-matching must use either all the grey triples (corresponding to x_i = true) or all the blue ones (corresponding to x_i = false)



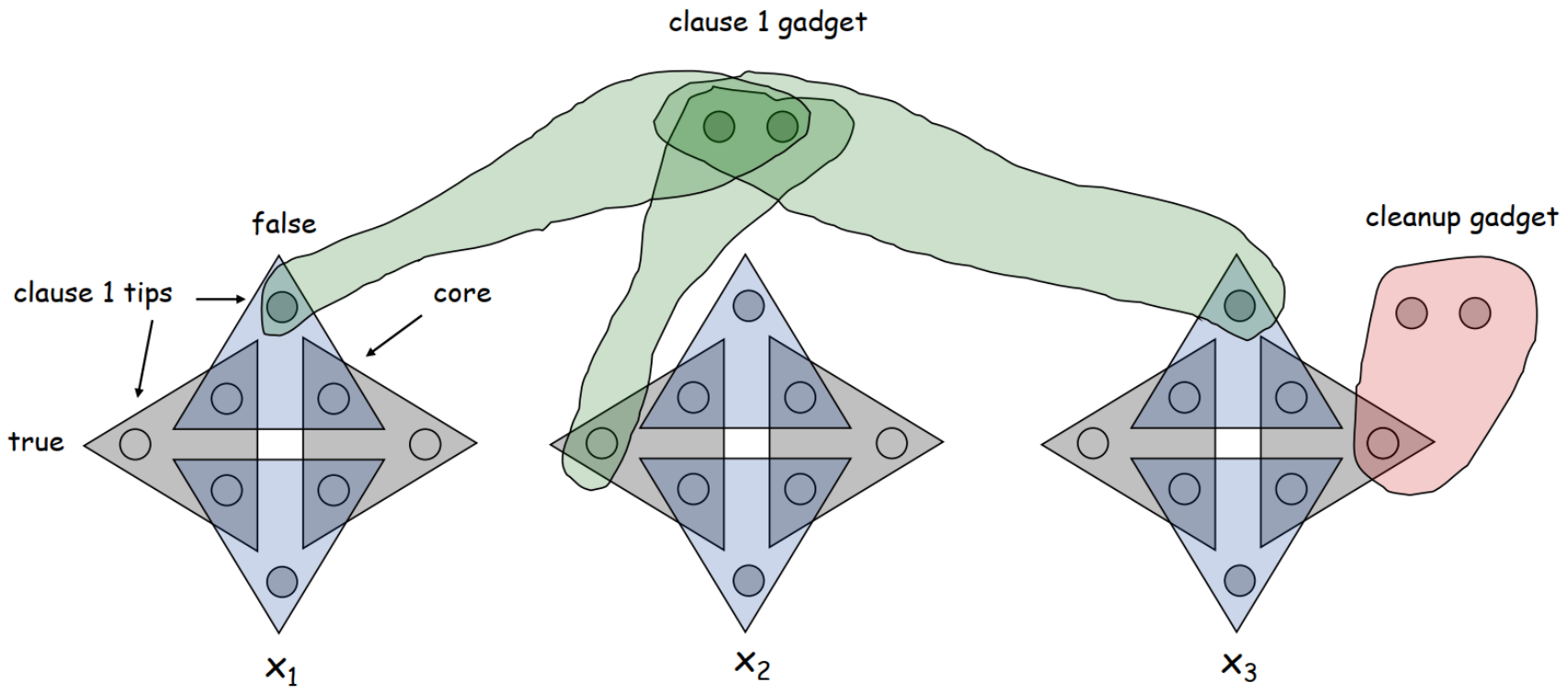
3-Dimensional Matching

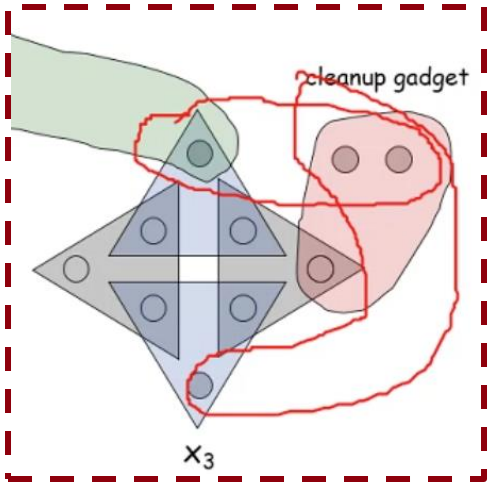
- **Construction. (part 2)**
  - For each clause C_j create gadget with two elements and three triples
  - Exactly one of these triples will be used in any 3D-matching
  - Ensures any 3D-matching uses either (i) grey core of x_1 or (ii) blue core of x_2 or (iii) grey core of x_3



3-Dimensional Matching

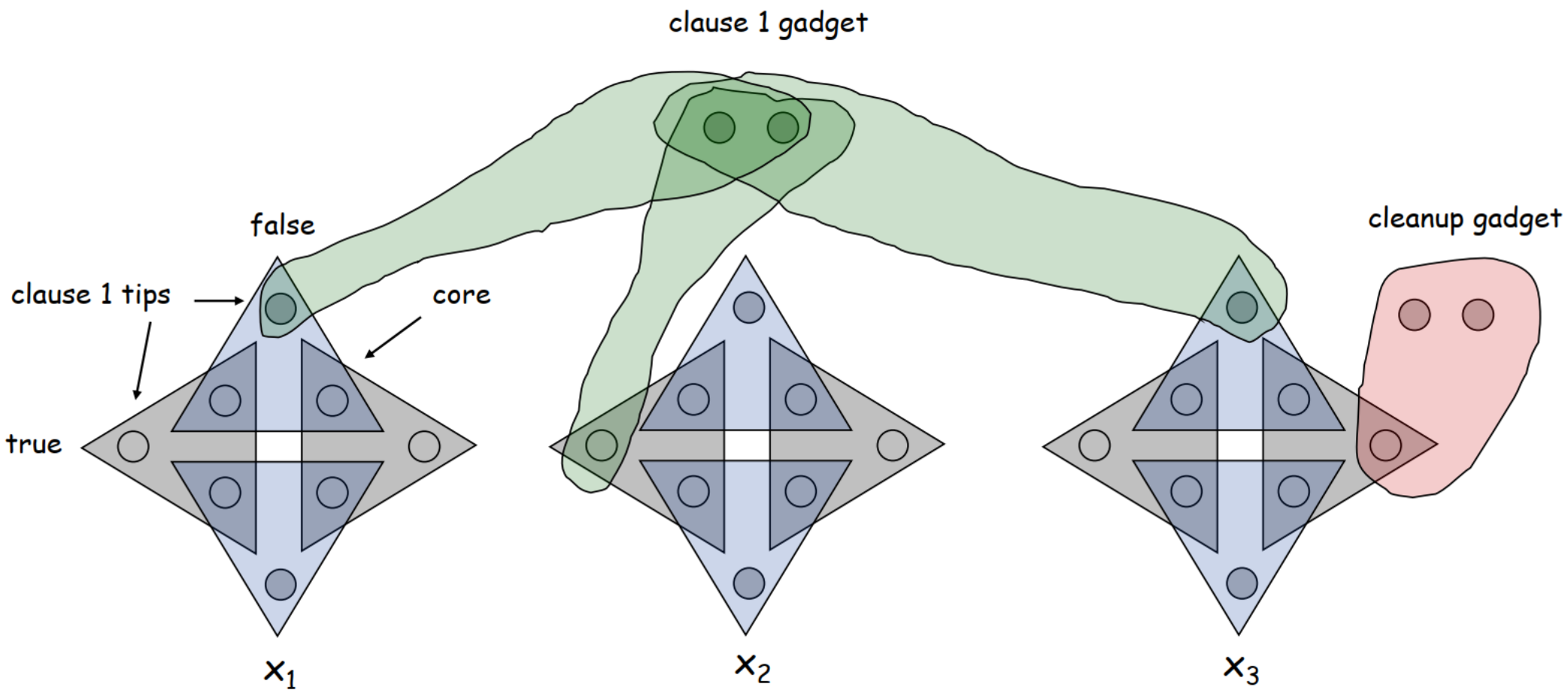
- **Construction (part 3)**
  - There are 2nk tips. nk covered by blue/gray triples, k covered by clause triples
  - To cover the remaining (n-1)k tips, create (n-1)k cleanup gadgets, each connected to all the 2nk tips





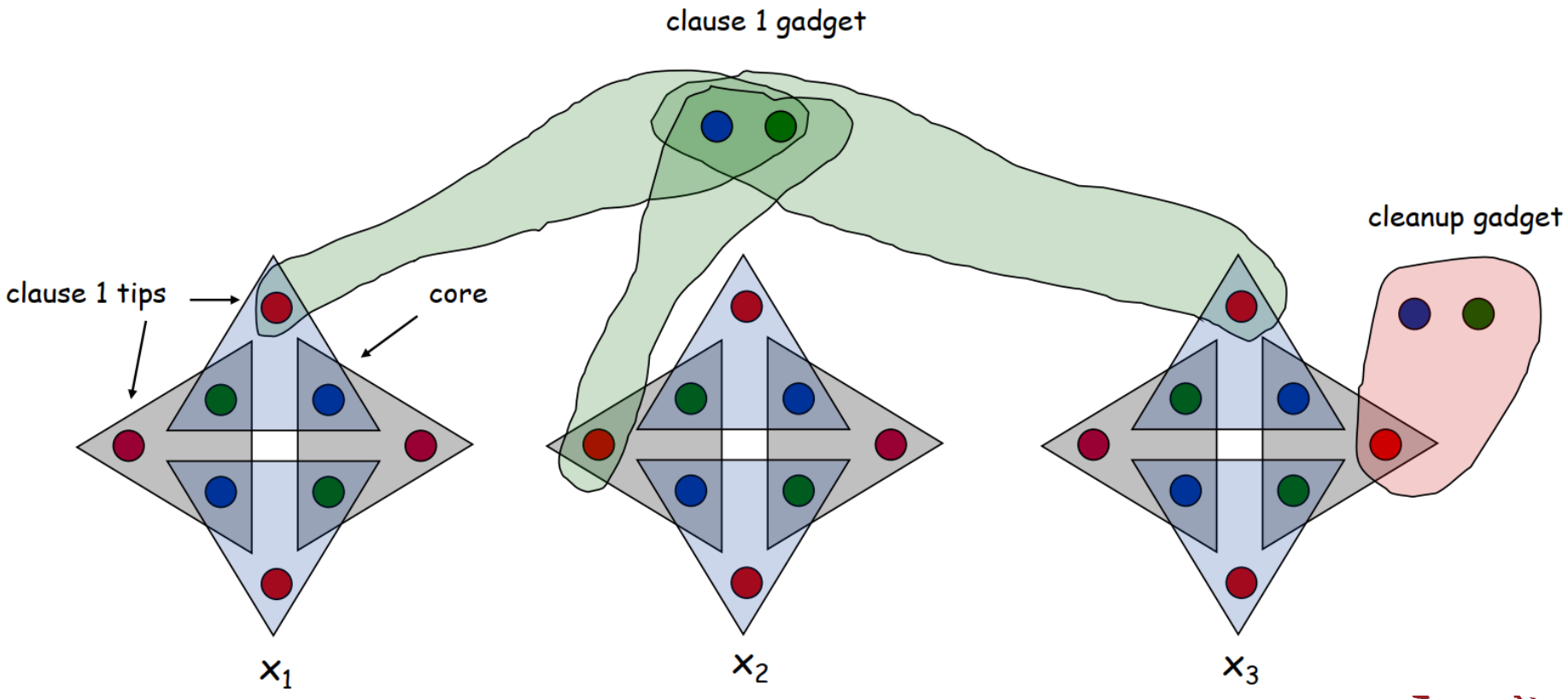
3-Dimensional Matching

- **Claim.** Instance has a 3D-matching iff ϕ is satisfiable
- **Q.** What are X, Y, and Z? Does each triple contain one element from each of X, Y, Z?



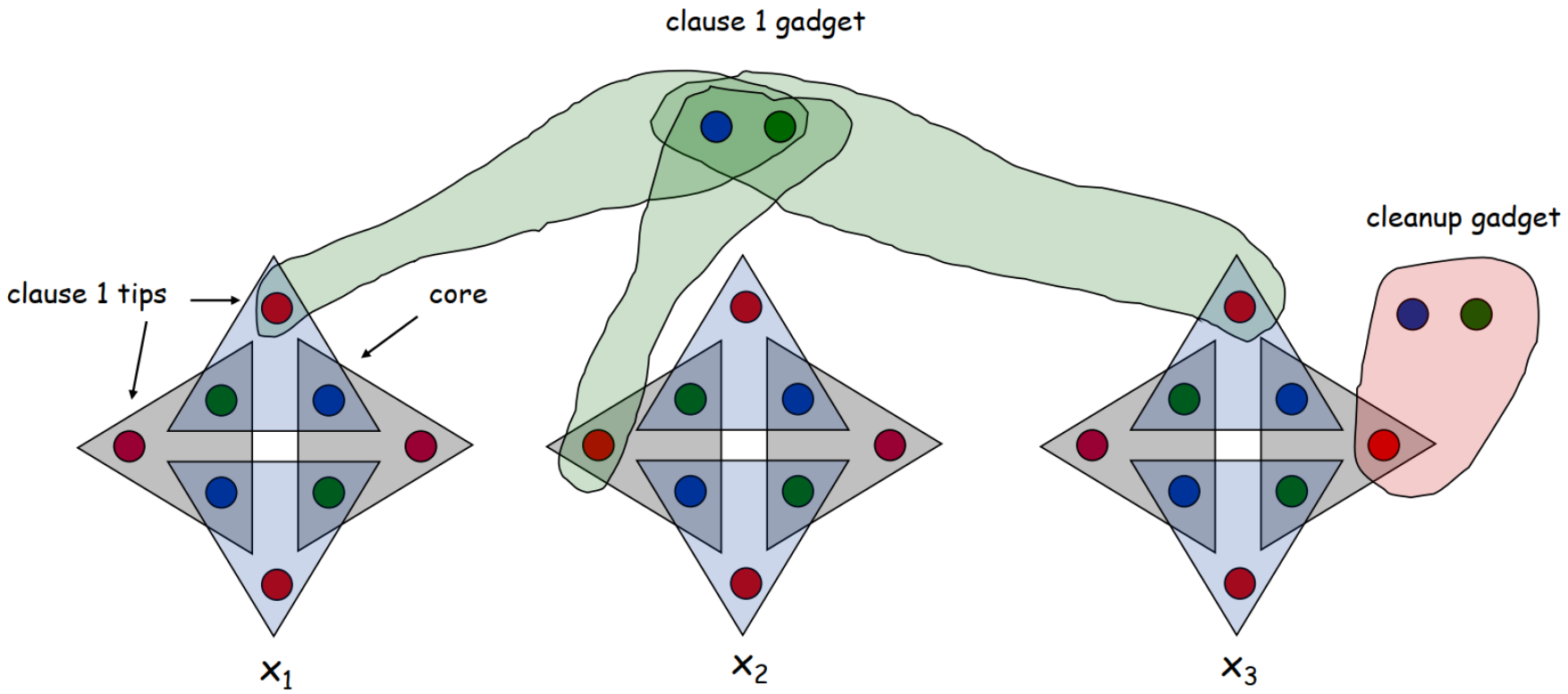
3-Dimensional Matching

- **Claim.** Instance has a 3D-matching iff ϕ is satisfiable
- **Q.** What are X, Y, and Z? Does each triple contain one element from each of X, Y, Z?
- **A.** X = red, Y = Green, Z = Blue



3-Dimensional Matching

- **Claim.** Instance has a 3D-matching iff ϕ is satisfiable
- **Pf. →** If 3D-matching, then assign x_i according to the color of the selected triples in gadget x_i
- **Pf. ←** If ϕ is satisfiable, then use assignment of x_i to select gadget x_i triples and use true literal in C_j to select gadget C_j triple



Numerical Problems

- Basic genres
  - Packing problems: SET-PACKING, INDEPENDENT SET
  - Covering problems: SET-COVER, VERTEX-COVER
  - Constraint satisfaction problems: SAT, 3-SAT
  - Sequencing problems: HAMILTONIAN-CYCLE, TSP
  - Partitioning problems: 3D-MATCHING, 3-COLOR
  - **Numerical problems: SUBSET-SUM, KNAPSACK**

Subset Sum

- **SUBSET-SUM.** Given natural numbers $w_1, \ldots, w_n$ and an integer W, is there a subset that adds up to exactly W?

- **Ex:** { 1, 4, 16, 64, 256, 1040, 1041, 1093, 1284, 1344 }, W = 3754
- **Yes.** 1 + 16 + 64 + 256 + 1040 + 1093 + 1284 = 3754

- **Remark.** With arithmetic problems, input integers are encoded in binary. Polynomial reduction must be polynomial in binary encoding

3-SAT reduces to SUBSET-SUM

- **Claim.** 3-SAT $\leq_p$ SUBSET-SUM

- **Pf.** Given an instance ϕ of 3-SAT, we construct an instance of SUBSET-SUM that has solution iff ϕ is satisfiable

3-SAT reduces to SUBSET-SUM

- **Construction. Given 3-SAT instance ϕ** with n variable and k clauses, form 2n + 2k decimal integers, each of n + k digits, as illustrated below
  - One digit for each variable and for each clause
  - Two numbers for each variable
  - Two numbers for each clause
  - Sum of each variable digit is 1;
  - Sum of each clause digit is 4

$$C_1 = \bar{x} \vee y \vee z$$
$$C_2 = x \vee \bar{y} \vee z$$
$$C_3 = \bar{x} \vee \bar{y} \vee \bar{z}$$

dummies to get clause columns to sum to 4

	x	y	z	C_1	C_2	C_3	
x	1	0	0	0	1	0	100,010
¬ x	1	0	0	1	0	1	100,101
y	0	1	0	1	0	0	10,100
¬ y	0	1	0	0	1	1	10,011
z	0	0	1	1	1	0	1,110
¬ z	0	0	1	0	0	1	1,001
	0	0	0	1	0	0	100
	0	0	0	2	0	0	200
	0	0	0	0	1	0	10
	0	0	0	0	2	0	20
	0	0	0	0	0	1	1
	0	0	0	0	0	2	2
W	1	1	1	4	4	4	111,444

3-SAT reduces to SUBSET-SUM

- **Claim.** ϕ is satisfiable iff there exists a subset that sums to W
- **Pf.** → Suppose ϕ is satisfiable
  - Choose numbers corresponding to each true literal
  - Since ϕ is satisfiable, each clause digit sums to at least 1 from the chosen numbers
  - Choose dummy numbers to make each clause digit sum to 4

$$C_1 = \bar{x} \vee y \vee z$$
$$C_2 = x \vee \bar{y} \vee z$$
$$C_3 = \bar{x} \vee \bar{y} \vee \bar{z}$$

	x	y	z	C_1	C_2	C_3	
x	1	0	0	0	1	0	100,010
¬ x	1	0	0	1	0	1	100,101
y	0	1	0	1	0	0	10,100
¬ y	0	1	0	0	1	1	10,011
z	0	0	1	1	1	0	1,110
¬ z	0	0	1	0	0	1	1,001
dummies to get clause columns to sum to 4	0	0	0	1	0	0	100
	0	0	0	2	0	0	200
	0	0	0	0	1	0	10
	0	0	0	0	2	0	20
	0	0	0	0	0	1	1
	0	0	0	0	0	2	2
W	1	1	1	4	4	4	111,444

3-SAT reduces to SUBSET-SUM

- **Claim.** ϕ is satisfiable iff there exists a subset that sums to W
- **Pf.** ← Suppose there is a subset that sums to W
  - Each variable digit forces the subset to choose exactly one from x and ¬x
  - Each clause digit forces the subset to choose at least one literal in the clause
  - Assign each variable according to the chosen literal

$$C_1 = \bar{x} \vee y \vee z$$
$$C_2 = x \vee \bar{y} \vee z$$
$$C_3 = \bar{x} \vee \bar{y} \vee \bar{z}$$

	x	y	z	C_1	C_2	C_3	
x	1	0	0	0	1	0	100,010
¬ x	1	0	0	1	0	1	100,101
y	0	1	0	1	0	0	10,100
¬ y	0	1	0	0	1	1	10,011
z	0	0	1	1	1	0	1,110
¬ z	0	0	1	0	0	1	1,001
dummies to get clause columns to sum to 4	0	0	0	1	0	0	100
	0	0	0	2	0	0	200
	0	0	0	0	1	0	10
	0	0	0	0	2	0	20
	0	0	0	0	0	1	1
	0	0	0	0	0	2	2
W	1	1	1	4	4	4	111,444

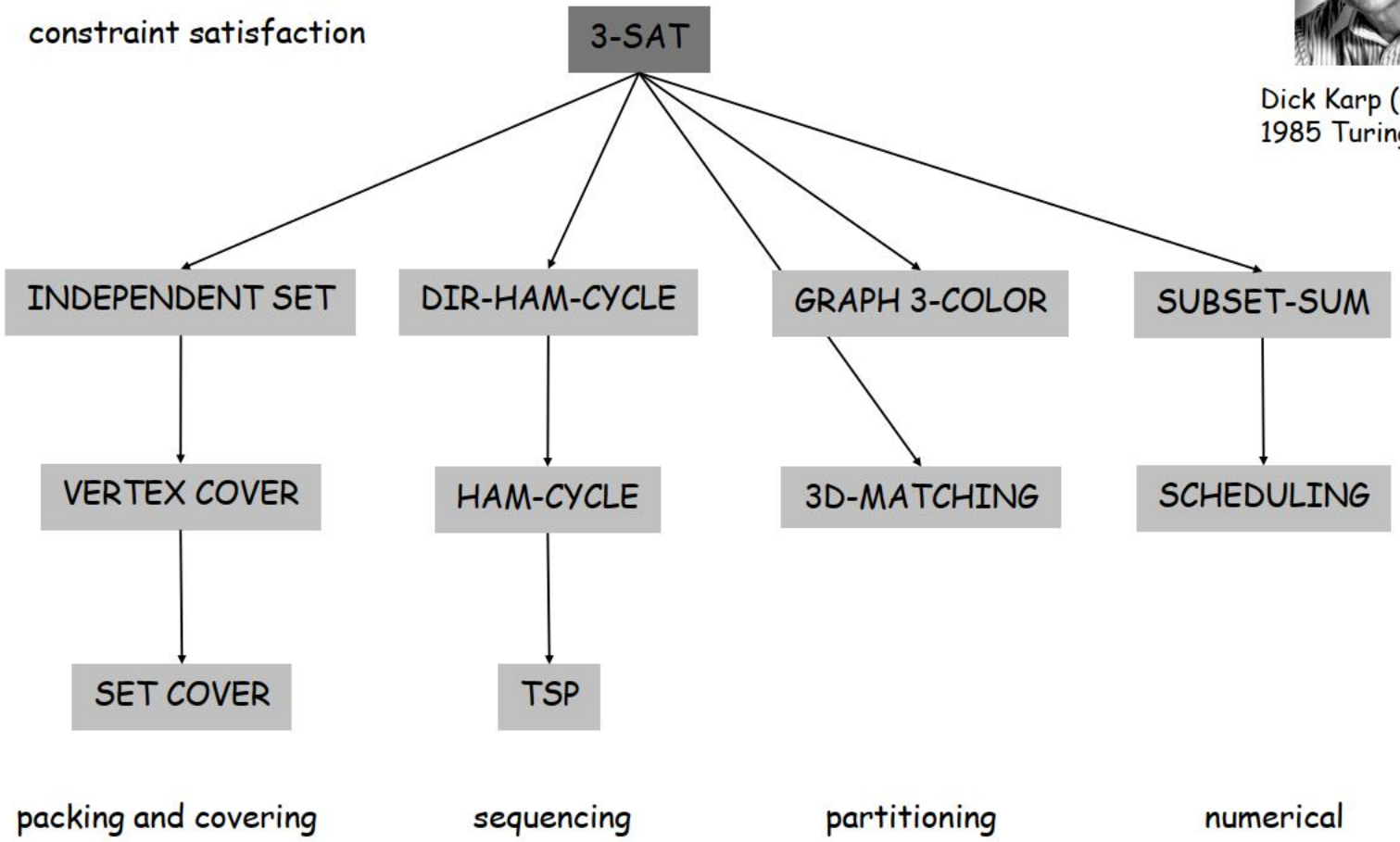
3D-MATCHING reduces to SUBSET-SUM

- **Construction.** Let $X \cup Y \cup Z$ be an instance of 3D-MATCHING with triplet set T. Let $n = |X| = |Y| = |Z|$ and $m = |T|$
- Let $X = \{ x_1, x_2, x_3, x_4 \}$, $Y = \{ y_1, y_2, y_3, y_4 \}$, $Z = \{ z_1, z_2, z_3, z_4 \}$
- For each triplet $t = (x_i, y_j, z_k) \in T$, create an integer w_t in base m+1 with 3n digits that has a 1 in position i, n+j, and 2n+k
- **Claim.** 3D-matching iff some subset sums to W = 111,…,111

Triplet t_i			x_1	x_2	x_3	x_4	y_1	y_2	y_3	y_4	z_1	z_2	z_3	z_4	w_i
x_1	y_2	z_3	1	0	0	0	0	1	0	0	0	0	1	0	100,001,000,010
x_2	y_4	z_2	0	1	0	0	0	0	0	1	0	1	0	0	10,000,010,100
x_1	y_1	z_1	1	0	0	0	1	0	0	0	1	0	0	0	100,010,001,000
x_2	y_2	z_4	0	1	0	0	0	1	0	0	0	0	0	1	10,001,000,001
x_4	y_3	z_4	0	0	0	1	0	0	1	0	0	0	0	1	100,100,001
x_3	y_1	z_2	0	0	1	0	1	0	0	0	0	1	0	0	1,010,000,100
x_3	y_1	z_3	0	0	1	0	1	0	0	0	0	0	1	0	1,010,000,010
x_3	y_1	z_1	0	0	1	0	1	0	0	0	1	0	0	0	1,010,001,000
x_4	y_4	z_4	0	0	0	1	0	0	0	1	0	0	0	1	100,010,001
															111,111,111,111

A Partial Taxonomy of Hard Problems

Polynomial-Time Reductions



Dick Karp (1972)
1985 Turing Award

Summary

Key Concepts

- **Decision problem**
  - Answer yes/no

- **P.** Decision problems for which there is a **poly-time** algorithm
- **NP.** Decision problems for which there exists a **poly-time** certifier
  - Algorithm C(s, t) is a **certifier** for problem X if for every string s, s ∈ X iff there exists a string t such that C(s, t) = yes

- **Co-NP.** Complements of decision problems in NP
- **EXP.** Decision problems for which there is an **exponential-time** algorithm
- **Claim.** P ⊆ NP, co-NP ⊆ EXP

Polynomial-Time Reduction

- **Reduction.** Problem X **polynomial-time reduces** to problem Y if arbitrary instances of problem X can be solved using:
  - Polynomial number of standard computational steps, plus
  - Polynomial number of calls to oracle that solves problem Y

- **Notation.** $X \leq_p Y$

- **Common approach: polynomial transformation**
  - Given any input x to X, **construct** an input y in poly-time such that x is a yes instance of X **iff** y is a yes instance of Y

NP-Completeness

- **NP-complete.** A problem Y in NP with the property that for every problem X in NP, $X \leq_p Y$

- **Recipe to establish NP-completeness of problem Y**
  - Step 1. Show that Y is in NP
  - Step 2. Choose an NP-complete problem X
  - Step 3. Prove that $X \leq_p Y$